FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 3

LLM Training

Zdeněk Kasner

3 Mar 2026

Training neural networks

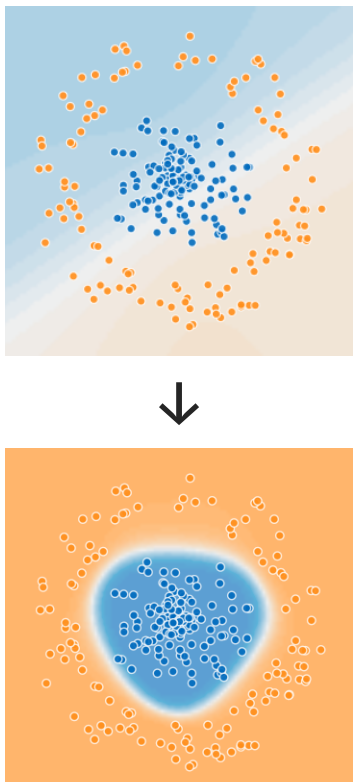
Training neural networks

source: https://playground.tensorflow.org

👩‍🍳 Recipe for supervised training of NNs

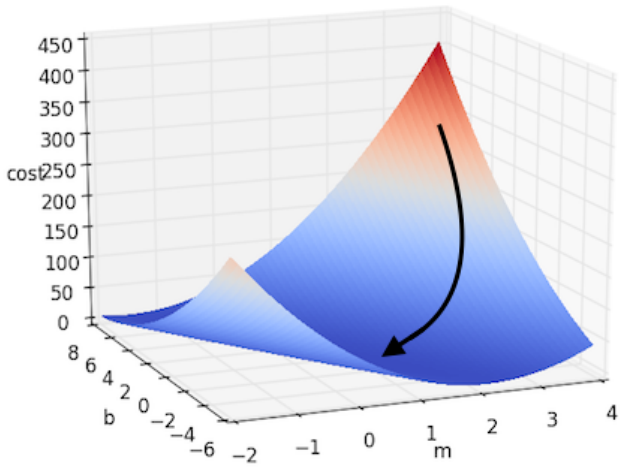
Repeat for each training example (x, y):

1. Let the model **predict** $\mathcal{M}(x, y) = \hat{y}$.
2. Compute a **loss function** $\mathcal{L}(\hat{y}, y)$: how wrong the model prediction $\hat{y}$ is.
3. Compute **gradients**: how each parameter of the model $\mathcal{M}$ contributed to the loss $\mathcal{L}$.
4. Apply **backpropagation**: update the model $\mathcal{M}$ parameters in the direction that reduces the loss $\mathcal{L}$.

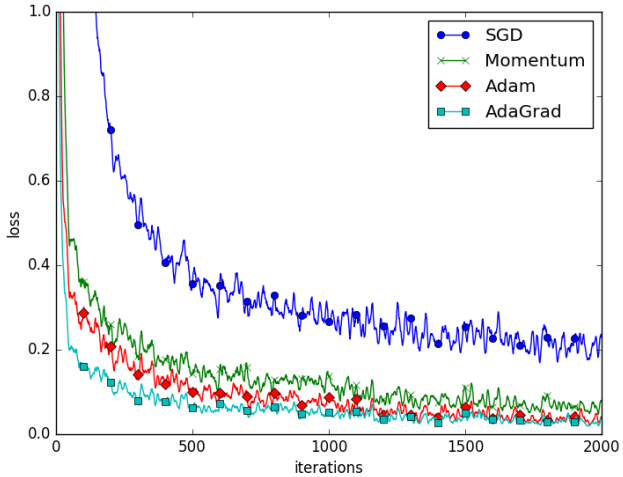


How does training a neural network look like?

source: https://playground.tensorflow.org



source: https://ml4a.github.io



source: https://kaeken.hatenablog.com/

Training a language model – closer look

source: Karpathy's microgpt

```python
# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.
```

```python
    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')
```

Loss function

A **loss function** (or cost function) measures **how far off** the model's prediction is from the correct answer.

How do we measure that? Depends on our problem.

> **Question**
>
> Assume your model predicts a **real number**. How would you measure the error with respect to the ground truth value?

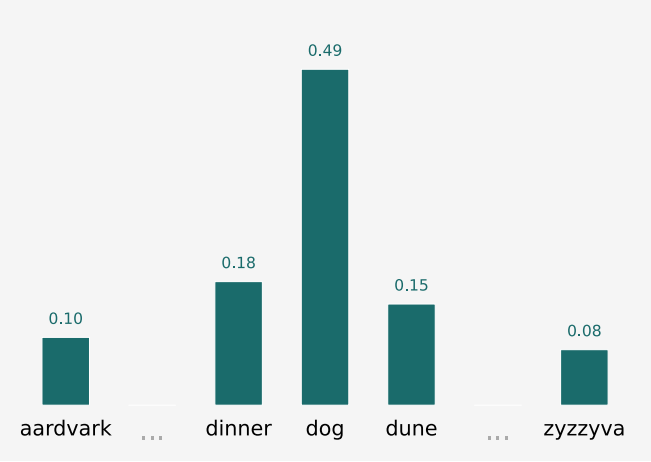
- `(true - predicted)`: bad idea, the errors may cancel out
- `abs(true - predicted)`: ok, but non-differentiable at 0 → training issues
- `(true - predicted)^2`: works well → smooth, penalizes large errors

Loss function: language modeling

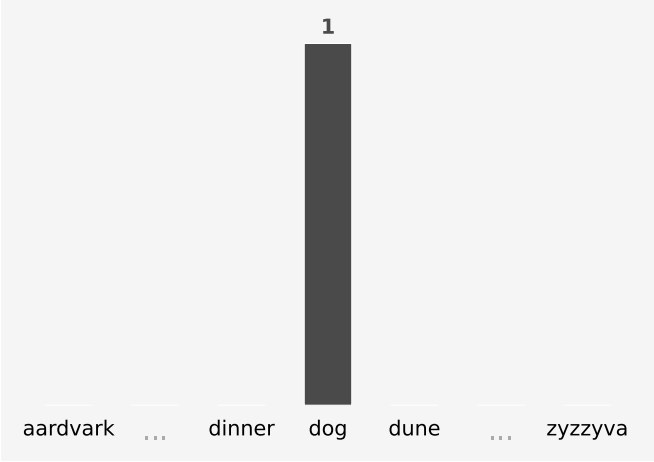
Question

Assume your model predicts a **probability distribution for the next word**. How would you measure the error with respect to the ground truth value?

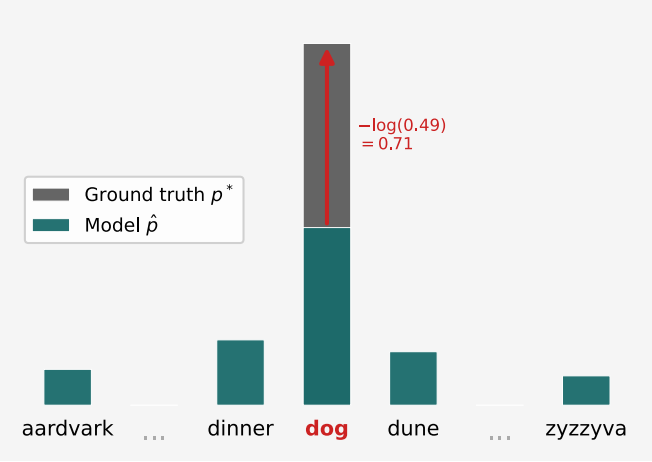
Model prediction



Ground truth



Model error



Loss function: language modeling

source: The Illustrated Transformer

What we train for:

Output Vocabulary:	a	am	I	thanks	student	<eos>
position #1	0.0	0.0	1.0	0.0	0.0	0.0
position #2	0.0	1.0	0.0	0.0	0.0	0.0
position #3	1.0	0.0	0.0	0.0	0.0	0.0
position #4	0.0	0.0	0.0	0.0	1.0	0.0
position #5	0.0	0.0	0.0	0.0	0.0	1.0

What we hope to get:

Output Vocabulary:	a	am	I	thanks	student	<eos>
position #1	0.01	0.02	0.93	0.01	0.03	0.01
position #2	0.01	0.8	0.1	0.05	0.01	0.03
position #3	0.99	0.001	0.001	0.001	0.002	0.001
position #4	0.001	0.002	0.001	0.02	0.94	0.01
position #5	0.01	0.01	0.001	0.001	0.001	0.98

Cross-entropy loss

The loss function used for the next word prediction task is called **cross-entropy**:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \cdot \log(\hat{y}_c)$$

where y_c is the true label (one-hot) and $\hat{y}_c$ is the predicted probability.

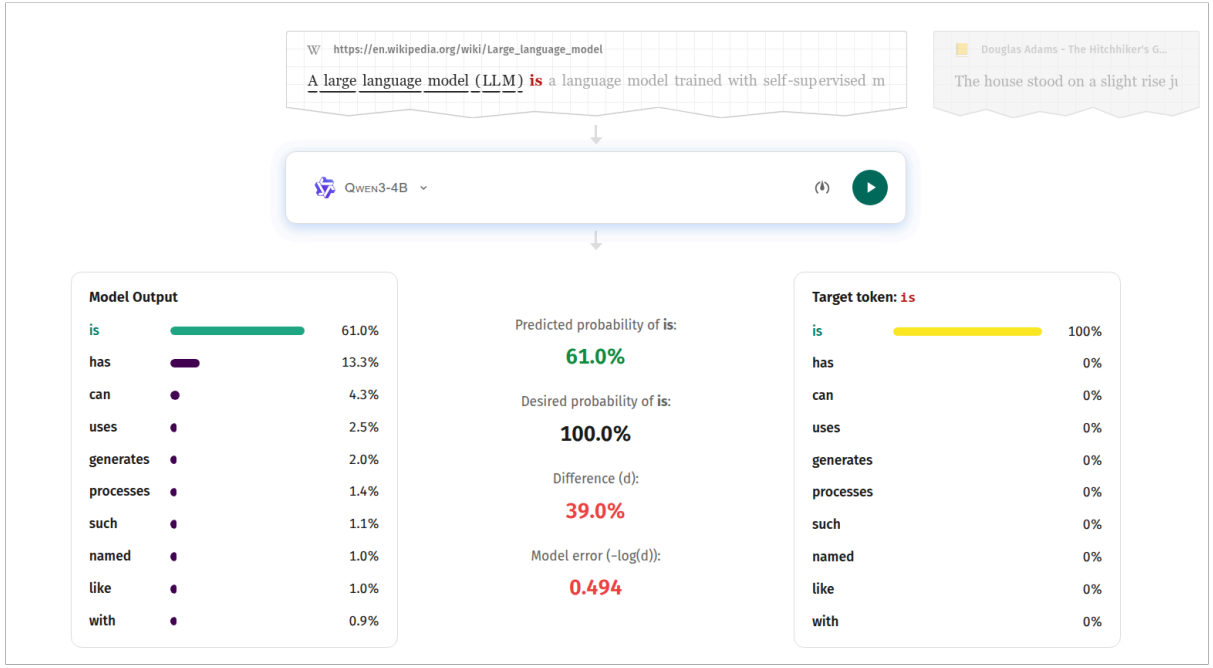
Intuition about cross-entropy for language modeling

- The model is supposed to predict the target token with **100% probability**.
- Any difference between these probabilities is considered as an **error**.
- The value of the loss is a **negative logarithm of the difference**. (why?)
- We ignore model predictions for any other tokens. (why?)

Cross-entropy in language models

Let's see it animated:

https://animatedllm.github.io/pretraining-simple



Descending the loss landscape

source: Hackernoon

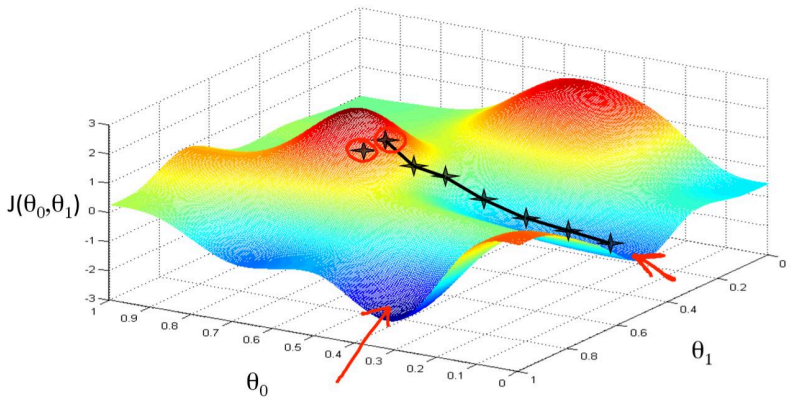
Our goal is to **minimize the loss** by iteratively **updating the model parameters** θ in the direction that reduces the loss $\mathcal{L}$:

Gradient descent as a math formula

$$\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$$

This simply says: we compute the steepest ascent direction (∇) and go the opossite way.

- ∇ is the **gradient** of the loss w.r.t. each parameter.
- η is the **learning rate**.

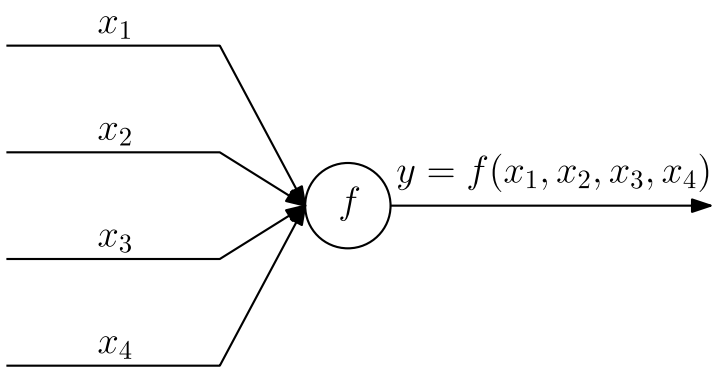


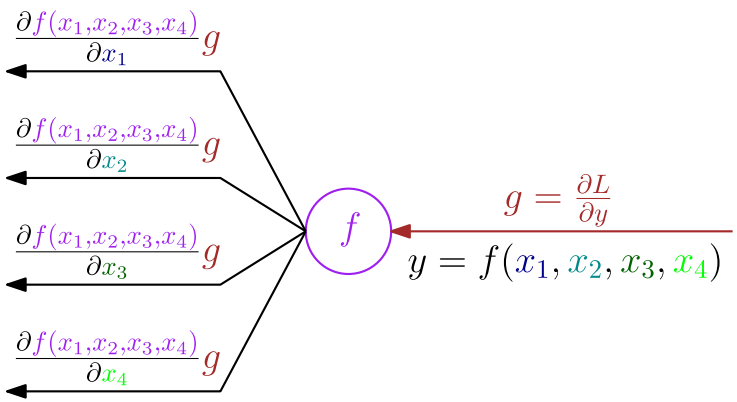
Computing gradient

source: NPFL138

Question

We ended the forward pass by computing the loss. How do we get the gradient?





We start by computing $\frac{\partial \mathcal{L}}{\partial y}$ → **the derivative of the loss with respect to the model's output probabilities** (the softmax outputs).

Computing gradient

Since our loss is computed as:

$$\mathcal{L} = -\log y_{\text{correct}},$$

then:

$$\frac{\partial \mathcal{L}}{\partial y_c} = \begin{cases} -\frac{1}{y_c} & \text{for the correct class} \\ 0 & \text{for all other classes} \end{cases}$$

Gradient vs. derivative

Gradient (∇) is how a (partial) **derivative** ∂ is called for multivariate functions (= functions operating on multi-dimensional inputs).

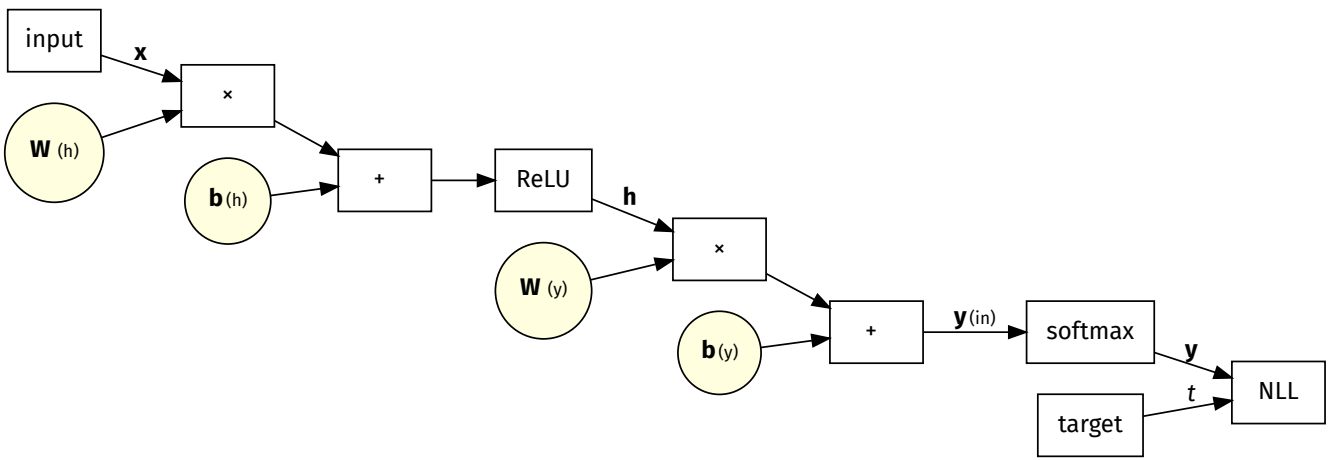
Backpropagation

source: NPFL129

Backpropagation = repeated application of the **chain rule** of derivatives for $f(g(x))$:

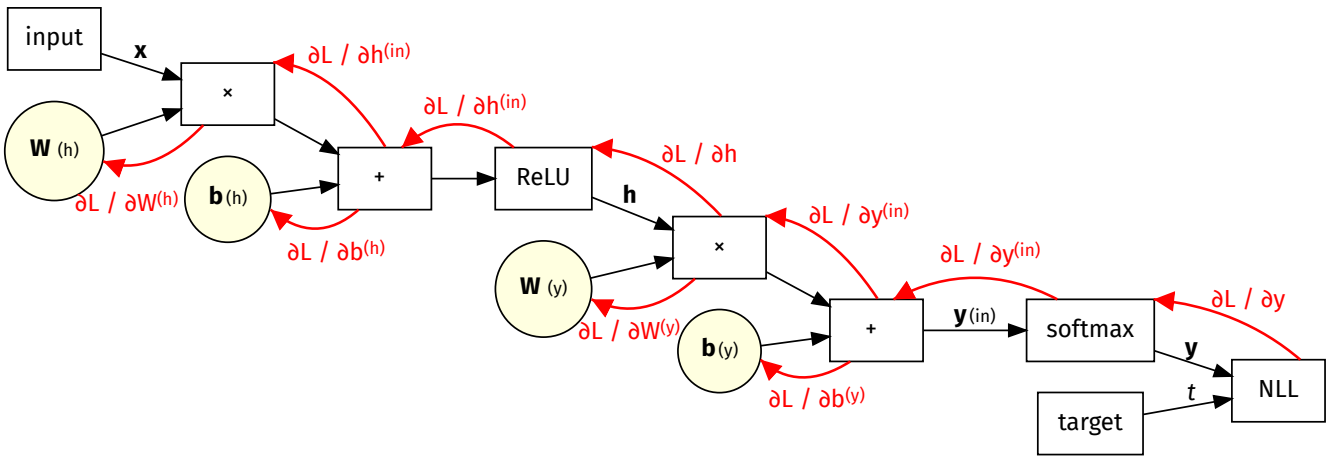
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

How to apply it? First, build a **computation graph** (example for a simple FFNN):



Backpropagation

...then apply the **chain rule** it in the backward direction:



Computing the gradient in practice

Modern deep learning frameworks (PyTorch, TensorFlow) have built-in **autograd** functionality that automatically computes gradients for us.

Stochastic gradient descent

Computing the gradient over the **entire dataset** at once is ~~expensive~~ impossible.

Idea

Instead of using all training examples, we iteratively compute the gradient on **small random subsets** of the training examples.

- **Stochastic gradient descent (SGD)**: our subset is of size 1 (=a single example).
- **Mini-batch SGD**: we a take a batch of examples (e.g. 32 examples).

Question

Is it useful to introduce some stochasticity into the training process?

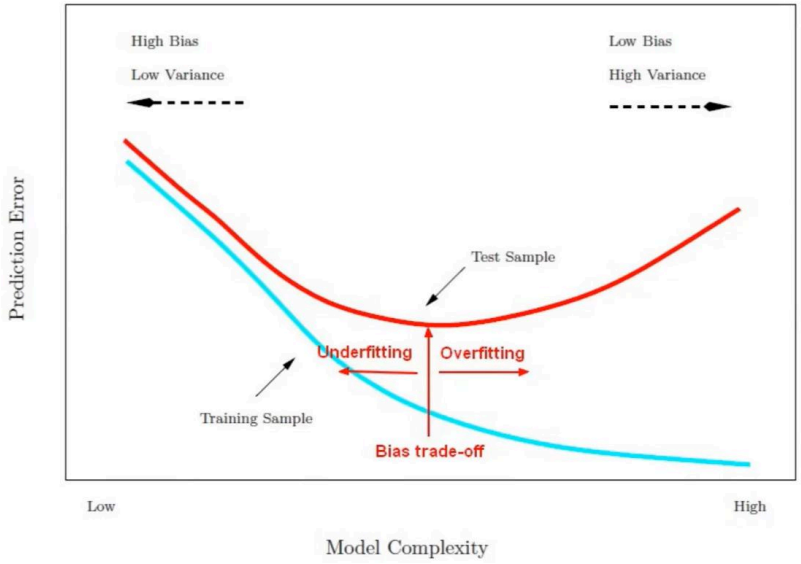
Learning rate

source: CS231n

The **learning rate** η controls how big the parameter update steps are.

Intuition about learning rate

- **Too high** → may overshoot the minimum, training diverges.
- **Too low** → converges very slowly, may get stuck in a bad local minimum.
- It is usually a small multiplier: 0.01, 0.001, …



Into the depths of deep learning

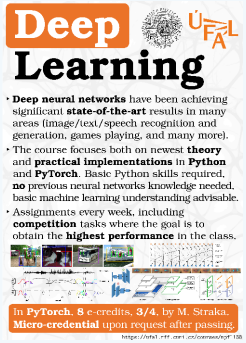
There are other concepts for NN training that we do not cover here, such as:

- **Weight initialization**: how do we initialize the model weights before training?
- **Regularization**: how do we prevent the model from overfitting to training data?
- **Optimizers**: is there are more efficient approach than SGD?

If you want to learn more...

See the MFF UK **Deep Learning course** (lectures 1-3, lecture recordings available):

https://ufal.mff.cuni.cz/courses/npfl138/2526-summer



Pretraining the Transformer

Supervised training?

source: Vaswani et al. (2017)

The original Transformer was trained **fully supervised on machine translation**:

- English → German (WMT 2014, 4.5M sentence pairs)
- English → French (WMT 2014, 36M sentence pairs)

> **Question**
>
> Where can we get that much data for other tasks?

We **do not have** that much data for other tasks 🥲

→ However, much of what the model was learning during the training was the **structure of the language** itself.

Self-supervised pretraining

Idea

If we train the model for predicting the words in a text, **any text is training data**!

W https://en.wikipedia.org/wiki/Large_language_model

A large **language** model (LLM) is a language model trained with self-supervised r

- The training regime where the labels come from the input itself is called **self-supervised learning**.
- We can use it for learning general language representations

  → ...maybe we will then need less data for the specific tasks?

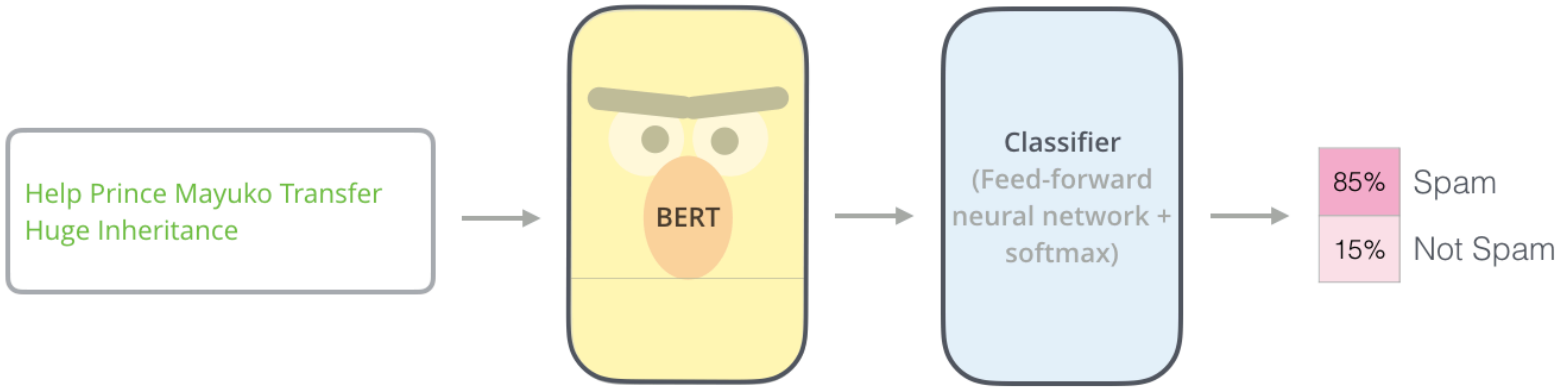
Pretraining a Transformer encoder

source: Illustrated BERT

Idea

We know the text representation is built in the Transformer **encoder**. Can we start pretraning the encoder alone?

Good representations might help us with classification tasks:

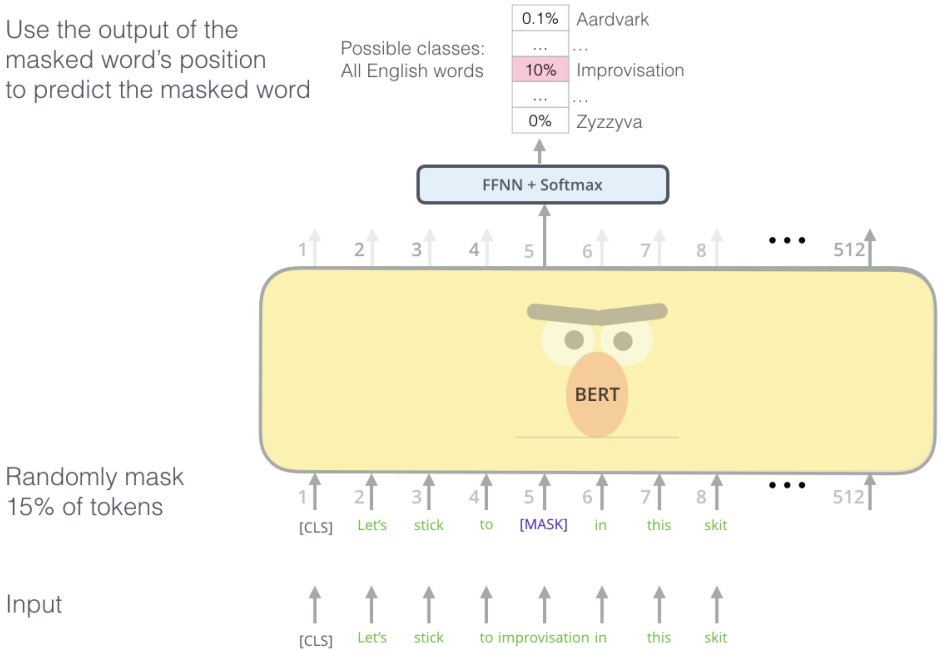


Pretraining a Transformer encoder

Our goal: Get a rich representation of the input text (one embedding per token).

- The resulting word embedding should incorporate the full sentence context (both **left and right**).
- Can we mask a word and let the model predict it?
  - Yes, but too slow → let's mask ~15% of words at a time.

→ the **masked language modeling** objective.



Pretraining a Transformer encoder

Our setup:

1 - Semi-supervised training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

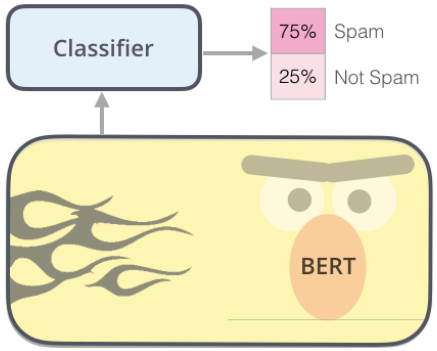
Model:

Dataset:

Objective: Predict the masked word (langauge modeling)

2 - Supervised training on a specific task with a labeled dataset.

Supervised Learning Step



Model: (pre-trained in step #1)

Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached…	Not Spam

BERT: Pre-trained Transformer encoder

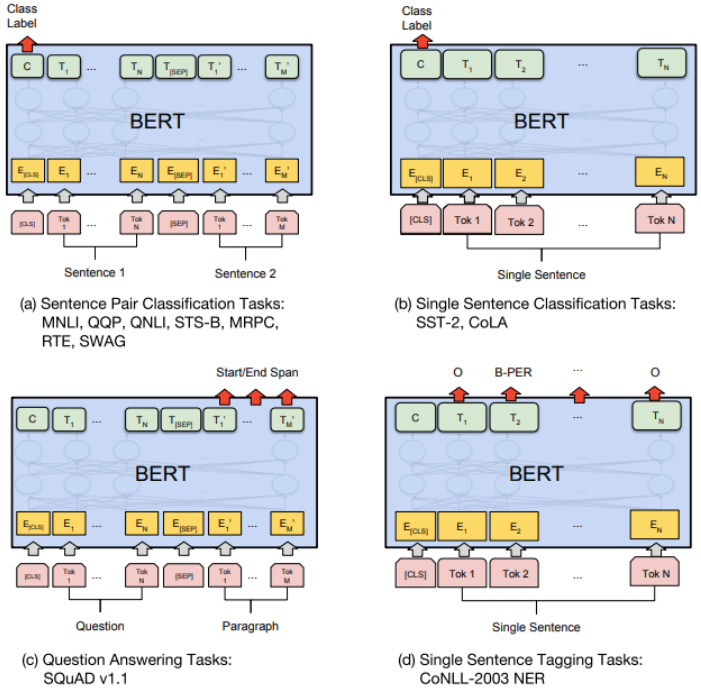
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

(Devlin et al., 2019) → breakthrough approach for many NLP tasks.

Info

The results BERT was getting were very cool – anything using BERT was suddenly **state-of-the-art**. No one was sure why.

This started a whole new research field called "BERTology".

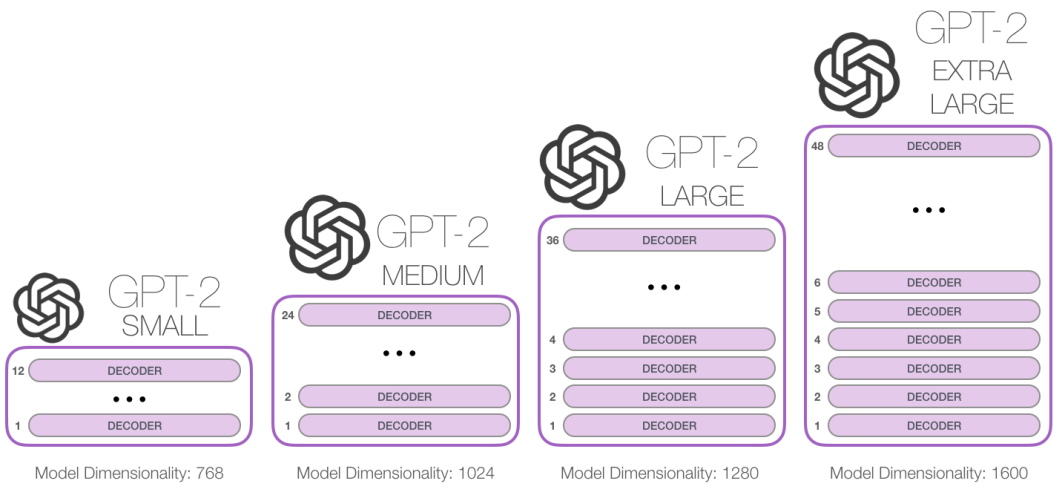


Pretraining a Transformer decoder

source: Illustrated GPT-2

Meanwhile in OpenAI...

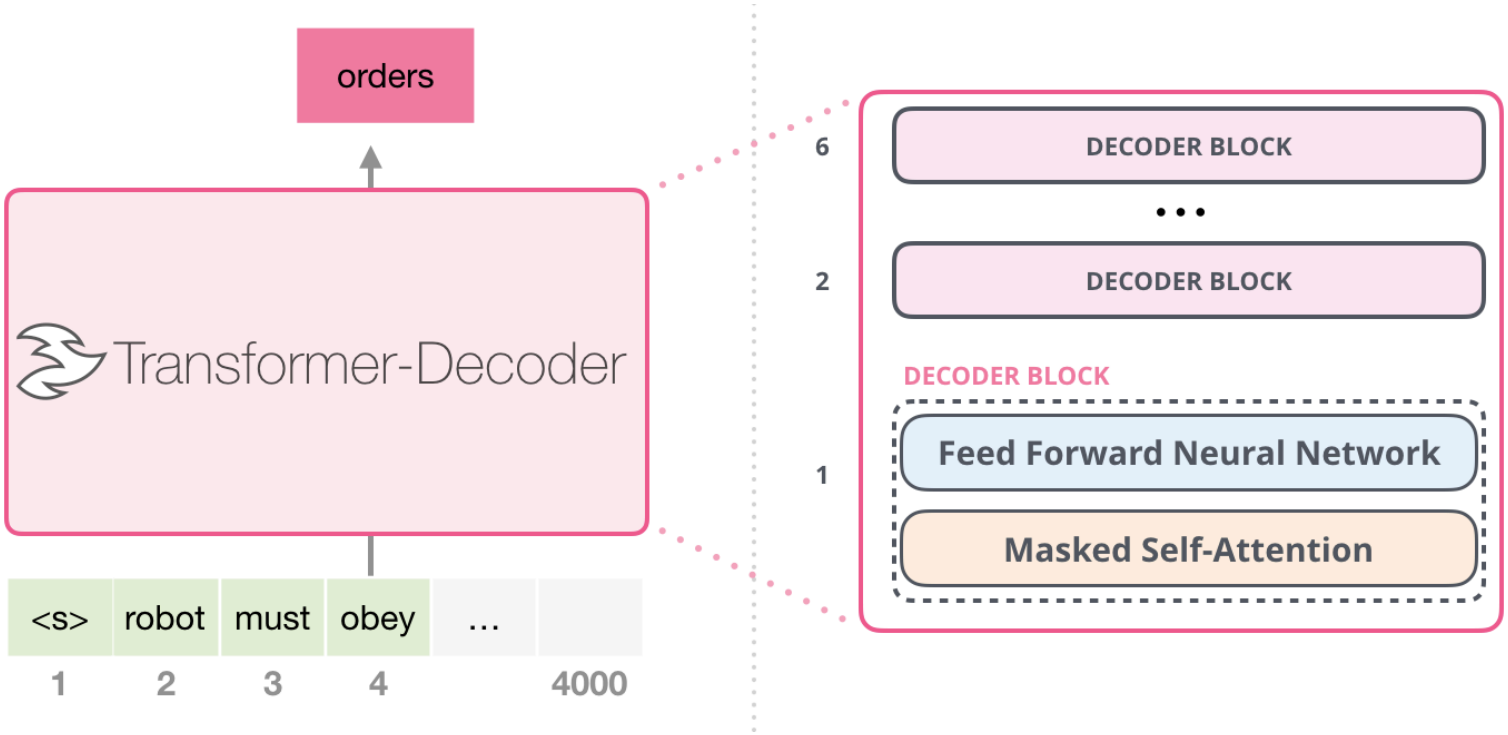
The encoder seems kind of useless. The **decoder** can represent the text on its own – and it can also **generate text**! Can we have more of that, please?



Pretraining a Transformer decoder

source: Illustrated GPT-2

Decoder-only block: only masked self-attention & FFNN, no encoder-decoder attention.



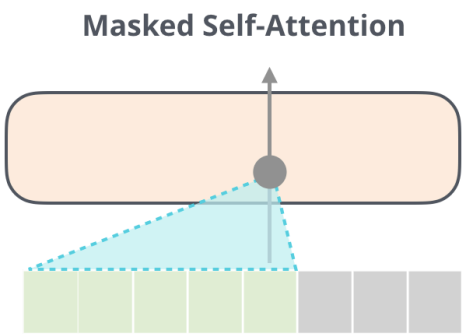
Causal language modeling

How to pretrain a decoder? Using **causal language modeling**: what we already know as "language modeling" or "next-word prediction task":

$$P(w_1, ..., w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, ..., w_{t-1})$$

Attention mask

Masking the future tokens is what allows **parallel training**: we can compute the loss at all positions at once, even though each position "pretends" it cannot see the future.



GPT-2: Pre-trained Transformer decoder

source: Radford et al. 2018

Generative Pre-trained Transformer

- Transformer **decoder only**.
- Pretrained with **causal language modeling** (next-word prediction).
- GPT-1 (2018): 12 layers, 117M params. Finetuned for downstream tasks.

GPT-2 (2019): scaled up to **1.5B params**, trained on WebText (8M web pages).

source: Radford et al. 2019

Key finding: the model can perform tasks **without any finetuning** – just from the pretraining data. This is what later became known as **zero-shot** capabilities.

Encoder-decoder pretraining

source: Raffel et al. 2020 (T5)

The encoder-decoder Transformer uses **span corruption** (denoising):

- Replace random **contiguous spans** with sentinel tokens (`<extra_id_0>`, `<extra_id_1>`, ...).
- The decoder generates only the **masked spans** (not the full text).

Example (T5):

Input: `"The <X> brown <Y> over the lazy dog"`

Target: `"<X> quick <Y> fox jumps"`

This combines the benefits of bidirectional encoding with autoregressive generation.

T5

source: Raffel et al. 2020

Text-to-Text Transfer Transformer

- **Encoder-decoder** architecture.
- All tasks framed as **text-to-text**: input text → output text.

Example:

- Translation: `"translate English to German: That is good"` → `"Das ist gut"`
- Summarization: `"summarize: ..."` → summary
- Classification: `"sentiment: great movie"` → `"positive"`

- Trained on C4 (Colossal Clean Crawled Corpus, 750GB of text).

- Various sizes: 60M to 11B parameters.
- Systematically compared many design choices (architecture, objectives, data).

GPT-3

source: Brown et al. 2020

175B parameters, trained on a mix of Common Crawl, books, Wikipedia.

Key contribution: **in-context learning** – the model can learn from a few examples provided in the input prompt, without any gradient updates.

Zero-shot

Task description only, no examples.

One-shot

Task description + one example.

Few-shot

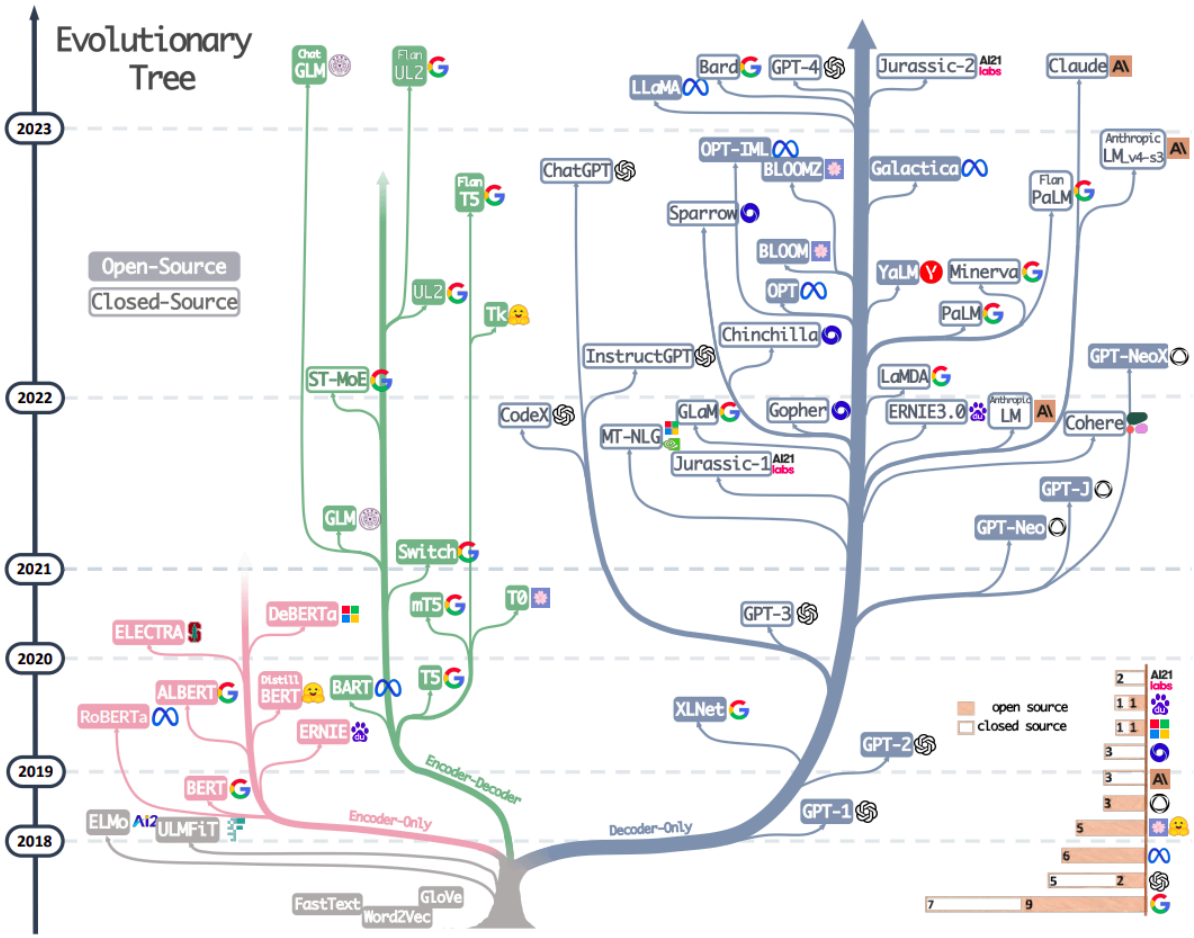
Task description + a few examples.

Info

> GPT-3 demonstrated that **scale alone** can unlock new capabilities. This was the starting point of the “scaling era” of LLMs.

The LLM evolutionary tree (up to 2023)

source: Yang et al. 2023



Legend:

- Encoder models
- Encoder-decoder models
- Decoder models

Finetuning – how it works

1. Start from a **pretrained checkpoint** (e.g. LLaMA-3-8B).
2. Prepare a dataset of **(input, output) pairs** for the target task.
3. Continue training with the **same objective** (cross-entropy on the output tokens).
4. Use a **lower learning rate** than pretraining (the model is already close to a good solution).

Why does this work?

The pretrained model has already learned rich language representations. Finetuning only needs to **slightly adjust** the parameters to fit the new task. This is much more data-efficient than training from scratch.

Instruction tuning

source: Wei et al. 2022 (FLAN)

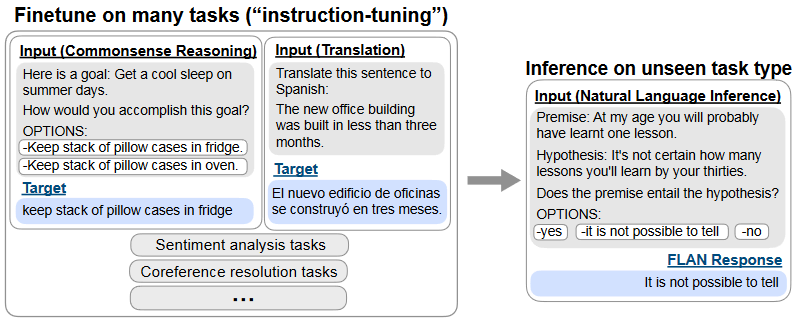
A special form of finetuning: train the model to **follow instructions**.

Training data format:

- **Instruction**: a natural language description of the task.
- **Input**: the actual input for the task (optional).
- **Output**: the expected response.

Example:

- Instruction: "Translate the following sentence to French."



- Input: “How are you?”
- Output: “Comment allez-vous?”

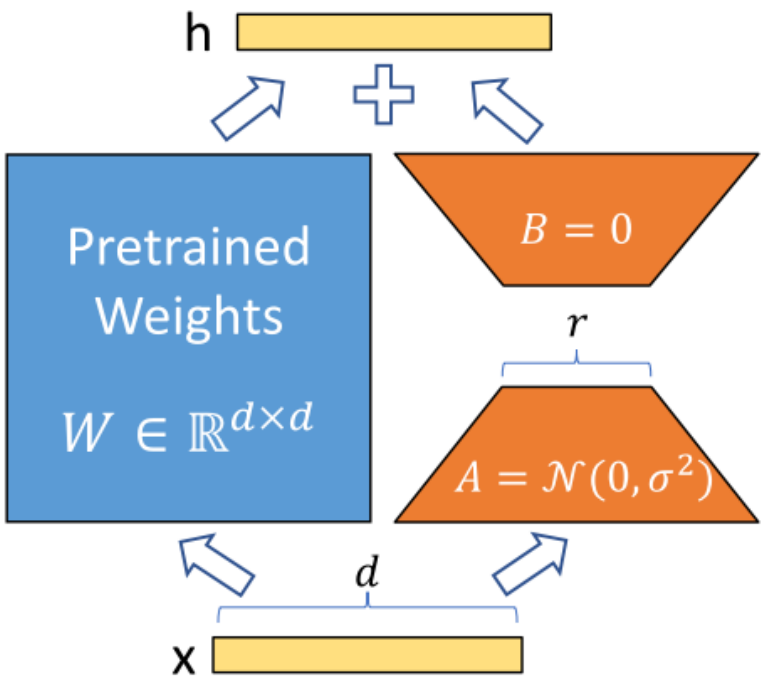
Parameter-efficient finetuning (PEFT)

source: Hu et al. 2021 (LoRA)

Finetuning **all parameters** of a large model is expensive. Can we finetune only **a few**?

LoRA (Low-Rank Adaptation):

- Add two small matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ parallel to each weight matrix.
- Only train A and B (where $r \ll d$).
- The update $\Delta W = BA$ is added to the original weights.
- Much fewer trainable parameters: $2 \times r \times d$ instead of d^2.



source: Dettmers et al. 2023

QLoRA: LoRA + 4-bit quantization → finetune a 65B model on a single GPU.

Alignment

The alignment problem

A pretrained (or instruction-tuned) model can generate fluent text, but it may still:

- **Hallucinate** facts.
- Produce **harmful or toxic** content.
- Be **unhelpful** or refuse reasonable requests.
- Not follow the user's **intent**.

Alignment

Make the model **helpful, harmless, and honest** – aligned with human preferences.

The key idea: use **human feedback** to teach the model what “good” output looks like, beyond what can be captured by next-token prediction.

LLM training pipeline

The standard training pipeline for modern LLMs has **three stages**:

1. Pretraining

- Self-supervised on massive text data.
- Learn general language knowledge.
- Trillions of tokens.
- Hundreds of GPUs for weeks/months.

2. Supervised finetuning

- Instruction-following data.
- Learn to be helpful and follow instructions.
- Thousands to millions of examples.
- Much cheaper than pretraining.

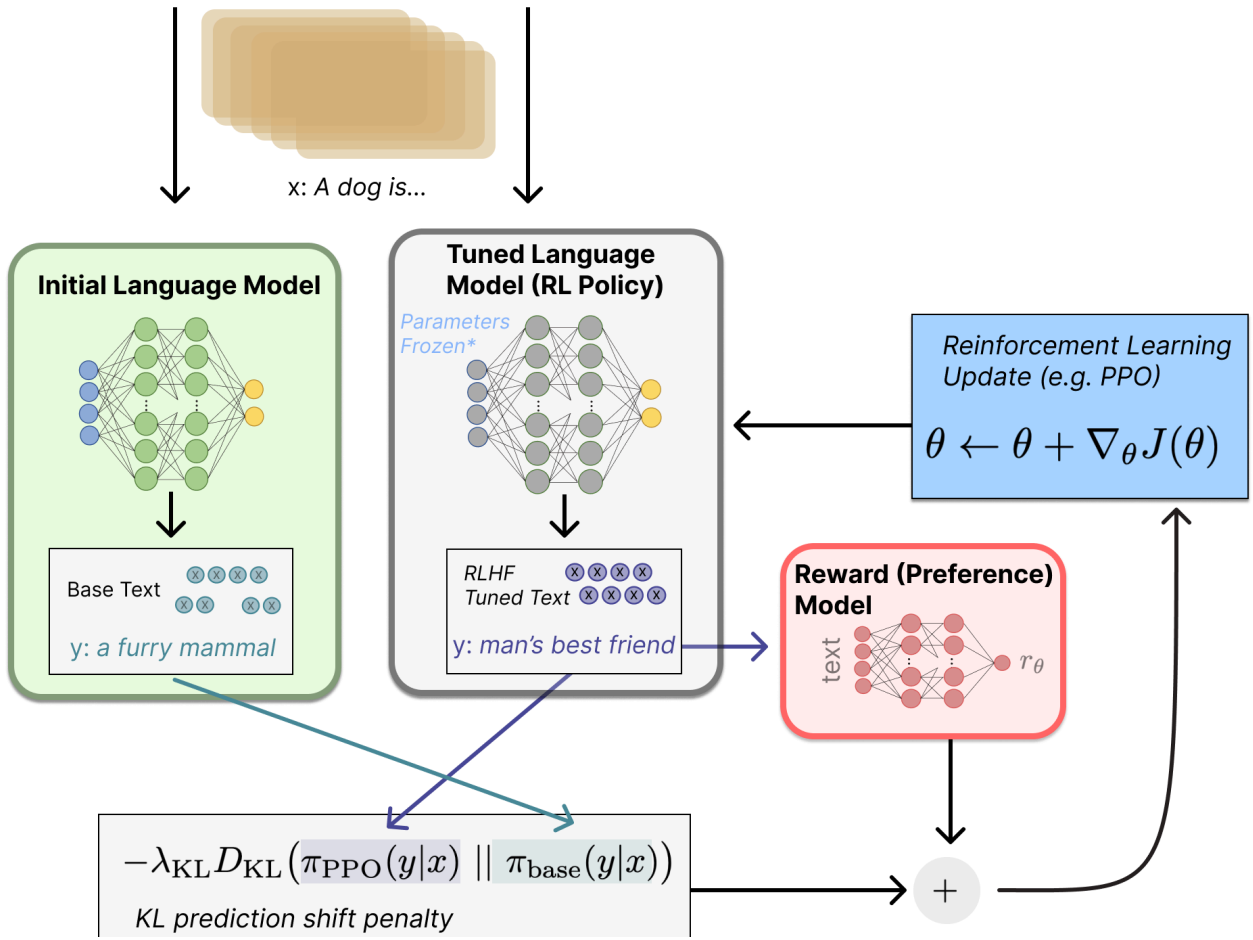
3. Alignment

- Human preference data.
- Learn to produce outputs humans prefer.
- RLHF, DPO, or variants.
- The “secret sauce” of ChatGPT.

Reinforcement learning from human feedback (RLHF)

Source: Ouyang et al. 2022 (InstructGPT)

RLHF was the key technique behind InstructGPT and ChatGPT.


x: A dog is...
Initial Language Model
Base Text
y: a furry mammal
Tuned Language Model (RL Policy)
Parameters Frozen*
RLHF Tuned Text
y: man's best friend
Reinforcement Learning Update (e.g. PPO)
$\theta \leftarrow \theta + \nabla_\theta J(\theta)$
Reward (Preference) Model
text
r_θ
$-\lambda_{\mathrm{KL}} D_{\mathrm{KL}}\left(\pi_{\mathrm{PPO}}(y|x) \;||\; \pi_{\mathrm{base}}(y|x)\right)$
KL prediction shift penalty
+

source: Hugging Face RLHF blog

RLHF – step by step

source: Ouyang et al. 2022

Step 1: Collect comparison data

- Generate multiple outputs for the same prompt.
- Have humans **rank** them (which output is better?).

Step 2: Train a reward model

- A separate model that takes (prompt, response) → score.
- Trained on human preference data.
- Learns to predict which response humans would prefer.

Step 3: Optimize the policy with RL

- Use the reward model as a signal.
- Optimize the LLM to produce outputs that get high reward.
- Usually with **PPO** (Proximal Policy Optimization).
- Include a **KL divergence penalty** to prevent the model from drifting too far from the SFT model.

$$\mathcal{L}_{\text{RLHF}} = -\mathbb{E}[r(x, y)] + \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

InstructGPT / ChatGPT

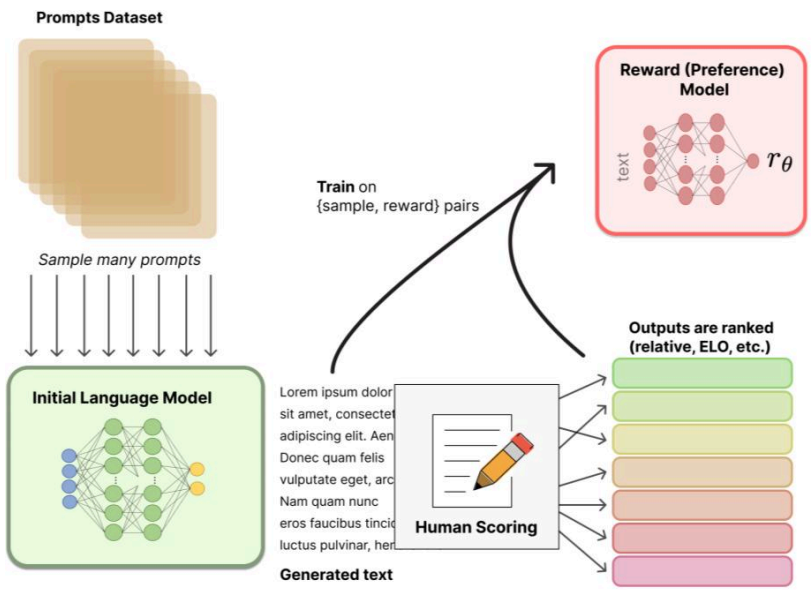
source: Ouyang et al. 2022

InstructGPT = GPT-3 finetuned with RLHF:

1. Start from GPT-3.
2. SFT on human-written demonstrations.
3. Train a reward model on human comparisons.
4. Optimize with PPO using the reward model.

Key result: InstructGPT (1.3B) was **preferred by humans** over the much larger GPT-3 (175B).

ChatGPT (Nov 2022) used the same approach, applied to GPT-3.5.



Direct preference optimization (DPO)

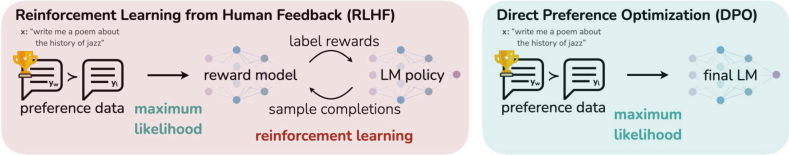
source: Rafailov et al. 2023

RLHF works but is **complex**: three models, unstable PPO, reward hacking. Can we do better?

DPO skips the reward model and RL entirely.

Given pairs of (preferred, dispreferred) responses,
DPO directly optimizes a **special loss function**:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$



where y_w = preferred response, y_l = dispreferred response.

- **Increases** probability of preferred responses.
- **Decreases** probability of dispreferred responses.
- The reference model π_{ref} prevents too large updates.

Summary

Summary

What we covered today

- **Training NNs**: loss functions, gradient descent, SGD, backpropagation, learning rate.
- **Pretraining**: self-supervised objectives (CLM, MLM, span corruption).
- **Pretrained models**: BERT, GPT, T5 and their variants; the LLM timeline.
- **Supervised finetuning**: adapting pretrained models to specific tasks; instruction tuning.
- **Alignment**: RLHF, DPO – making models helpful and safe.

Links

- BERT: Pre-training of deep bidirectional Transformers – Devlin et al. (2019)
- Language models are few-shot learners (GPT-3) – Brown et al. (2020)
- Exploring the limits of transfer learning (T5) – Raffel et al. (2020)
- Training language models to follow instructions (InstructGPT) – Ouyang et al. (2022)
- Direct preference optimization (DPO) – Rafailov et al. (2023)
- Illustrating RLHF – Hugging Face blog
- An overview of gradient descent optimization algorithms – Ruder (2016)
- The Illustrated BERT – Jay Alammar